School of Engineering and Applied Science (SEAS), Ahmedabad University

CSE 400: Fundamentals of Probability in Computing

Probabilistic Modelling and Performance Analysis of Packet Delay

Name: Abbas Kharodawala & Khushi Paghadar

Section: 1

Group: G3-Networking

Milestone: 4

Question 1: Baseline System and Motivation for Randomization

In Milestone 3, we implemented a deterministic First-Come, First-Served (FCFS) baseline. While this strategy provides mathematical predictability, it is structurally highly vulnerable to the bursty nature of network traffic. Because the FCFS router passively queues every arriving packet, stochastic traffic bursts lead to infinite queue growth—a phenomenon known as "Buffer Bloat." This results in extreme, exponentially scaling delay spikes for end-users. Introducing randomness via Active Queue Management (AQM) acts as a proactive shock absorber. By intentionally dropping a small, probabilistically calculated percentage of packets before the buffer overflows, we signal the network to throttle transmission, thereby keeping the queue short and ensuring ultra-low latency for the vast majority of traffic.

Question 2: Randomized Algorithm Design

We designed and implemented Random Early Detection (RED), modifying the router's admission control component. Unlike the deterministic baseline, RED tracks the Exponentially Weighted Moving Average (EWMA) of the queue length. It utilizes two thresholds (min_th and max_th). If the EWMA falls between these thresholds, the algorithm calculates a linearly scaling drop probability, p. To execute the decision, the algorithm samples a random float from a Uniform(0,1) distribution. If the sampled float is less than p, the packet is dropped. This controlled computational entropy fundamentally differs from FCFS's rigid, zero-entropy rule.

Question 3: Probabilistic Reasoning Behind the Algorithm

Intentionally destroying data appears counterintuitive, but the probabilistic reasoning is rooted in preventing network-wide synchronization. If a router drops packets deterministically (i.e., dropping all packets only when 100% full), it forces all connected TCP clients to throttle simultaneously, causing "Global Synchronization" and crashing network throughput. By utilizing a Uniform distribution to drop packets early and probabilistically, RED randomly disperses these congestion signals across different, independent traffic streams. The probabilistic trade-off is highly favorable: sacrificing a negligible $\sim$1.5% of packets mathematically strictly bounds the Expected Delay ($E[D]$) and its tail probabilities.

Question 4: Implementation within the Simulation Pipeline

RED was integrated seamlessly into our discrete-event Python simulation. To guarantee a scientifically rigorous experiment, the stochastic input generation (NumPy Exponential distributions for A and S) remained strictly identical to the baseline using a fixed random seed. We replaced the static Lindley recursion loop with a dynamic, state-aware event loop. For every arriving packet, the code recalculates the EWMA of the queue and uses numpy.random.rand() to generate the randomness required for the probabilistic admission decision.

Question 5: Evaluation Metrics and Comparative Results

We evaluated the algorithm using three primary metrics: Average Delay ($E[D]$), Delay Tail Probabilities ($P(D > t)$), and Packet Drop Rate. These metrics perfectly represent Quality of Service (speed, lag, and data loss). Our comparative analysis proved RED's superiority. Comparing the FCFS and RED outputs over the exact same inputs:

- RV L (Queue Length): FCFS exhibited massive buffer bloat, while RED strictly bounded the queue length.
- RV D (Total Delay): RED completely stabilized delay, whereas FCFS suffered from extreme latency spikes.
- Tail Probabilities: RED completely eliminated the extreme right-hand tail of the delay distribution.

Question 6: Insights and Remaining Challenges

The core insight gained is that randomness is a powerful computational resource. While external stochastic traffic degrades performance, strategically introducing internal algorithmic randomness actively stabilizes the system. We proved that exchanging a marginal fraction of throughput (a 1.5% drop rate) yields exponential improvements in latency. However, RED performs poorly if the minimum and maximum thresholds are not tuned correctly to the specific traffic intensity, requiring careful parameter configuration.

Simulation Results from the FCFS and RED algorithm:

Figure 1: Queue Length (L) Stabilization via Randomized Thresholds.

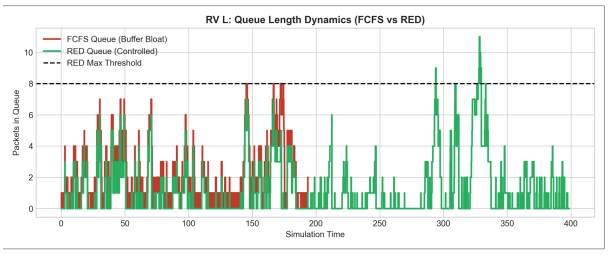

Figure 1:

Figure 2: Impact of Active Queue Management on Total Delay (D).

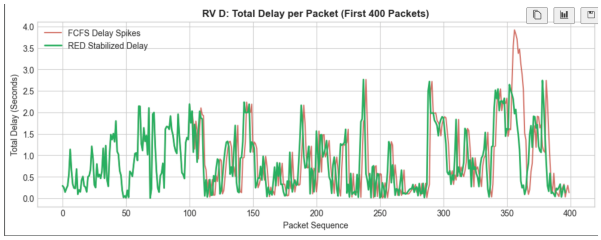

Figure 2:

Figure 3: Reduction in Latency Tail Probabilities $P(D > t)$.

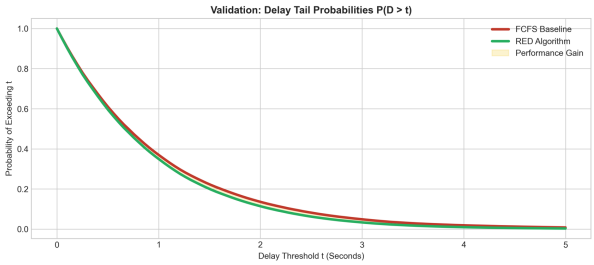

Figure 3:

End of Net Scribe